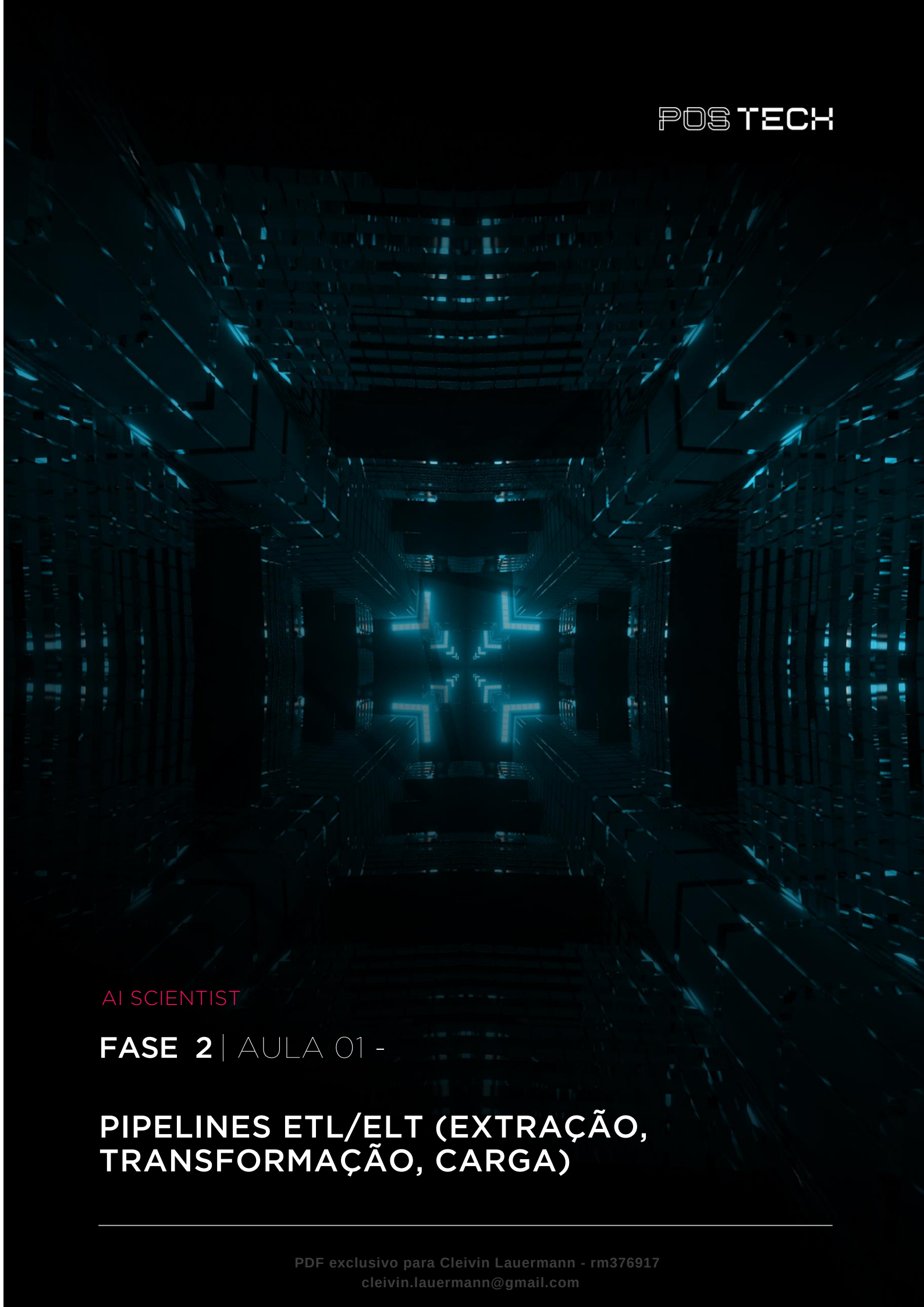


AI SCIENTIST

FASE 2 | AULA 01 -

PIPELINES ETL/ELT (EXTRAÇÃO, TRANSFORMAÇÃO, CARGA)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 7

MERCADO, CASES E TENDÊNCIAS 18

O QUE VOCÊ VIU NESTA AULA? 19

REFERÊNCIAS 21

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Para um AI Scientist, o modelo de Machine Learning é apenas a ponta do iceberg. A base que sustenta qualquer solução preditiva ou generativa é a engenharia de dados. Pipelines robustos garantem que o dado bruto — muitas vezes ruidoso e desestruturado — chegueem você de forma confiável.

Analistas e cientistas de dados chegam a gastar entre 60% e 80% do tempo apenas preparando e limpando dados (CASTELLANOS; TORRES, 2021). Portanto, entender como esses dados são movidos e transformados não é apenas "trabalho de engenheiro(a)", mas um requisito para agilizar o ciclo de vida dos seus modelos.

Historicamente, a arquitetura predominante para essa movimentação foi o **ETL** (*Extract, Transform, Load*), mas recentemente, impulsionado pela nuvem, consolidou-se o paradigma **ELT** (*Extract, Load, Transform*). Vamos entender as diferenças e aplicações de cada um.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Laboratório Prático: Pipeline ETL com Validação de Qualidade

Vamos construir um pipeline ETL simples, utilizando o Google Colab. O objetivo é compreender, de forma prática, como dados são ingeridos, tratados, validados e disponibilizados para consumo analítico seguindo a arquitetura medalhão.

Contexto do Problema

Você recebeu um arquivo CSV contendo informações de clientes e pedidos de um sistema transacional. Como ocorre em cenários reais, esses dados apresentam inconsistências, valores faltantes, duplicidades e erros de preenchimento. Seu papel é garantir que apenas dados confiáveis avancem no pipeline, preservando rastreabilidade, qualidade e governança.

Etapas do Laboratório

1. Configuração do Ambiente

Antes de iniciar o processamento, prepararemos o ambiente instalando as bibliotecas necessárias (`pandas`, `faker`, `fastparquet`) e criando a estrutura de pastas que representará nosso Data Lake (`raw`, `bronze`, `silver`, `gold`).

2. Geração dos Dados (RAW)

O pipeline inicia com a criação de um dataset sintético que simula dados reais, injetando propositalmente erros comuns (nulos, datas futuras, etc).

> **Desafio da etapa:** Como evolução, não utilize a função de geração e substitua a fonte de dados real. As opções incluem o consumo de uma API pública, a utilização de datasets abertos ou um dataset próprio.
>
> *Fontes sugeridas:* Base dos Dados, Google Dataset Search, Kaggle.

3. Ingestão na Camada Bronze

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Na camada Bronze, os dados são ingeridos exatamente como chegam da fonte, sem aplicação de regras de negócio (limpeza). O arquivo é lido, convertido para formato colunar (**Parquet**) para performance e enriquecido com metadados de ingestão (timestamp).

> **Desafio da etapa:** Implemente o armazenamento particionado por data de ingestão. Outra evolução proposta é persistir os dados fora do ambiente local do Colab, utilizando armazenamento como serviço (S3 ou Google Cloud Storage), reforçando o desacoplamento entre processamento e armazenamento.

4. Validação de Qualidade e Camada Silver

Nesta etapa, aplicamos regras de qualidade (ex: campos obrigatórios, validação de e-mail, datas lógicas).

- **Dados Válidos:** seguem para a pasta `silver` (limpos e deduplicados).
- **Dados Inválidos:** são isolados em uma `quarentena` para auditoria.

> **Desafio da etapa:** Classifique as regras por severidade (Críticas vs. Alerta). Enriqueça a tabela de quarentena com uma coluna explicando exatamente qual regra foi violada. Opcionalmente, utilize frameworks de validação de mercado como **Great Expectations** ou **Soda** para substituir as validações manuais.

5. Camada Gold (Agregação)

Os dados da Silver são transformados em KPIs de negócio (ex: Receita por Estado, Performance por Categoria). Esta camada entrega o dado pronto para ferramentas de BI e Modelos de IA.

> **Desafio da etapa:** Crie uma tabela orientada a Ciência de Dados com features agregadas por cliente (RFM – Recency, Frequency, Monetary). Tente migrar o armazenamento final desta camada para um Data Warehouse em nuvem, como BigQuery ou Athena.

Desafio Extra: Escalabilidade com Spark

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Imagine que o volume de dados subiu exponencialmente e o Pandas não consegue mais processar os dados em memória RAM.

O Desafio: Refatore o processamento das camadas **Silver** e **Gold** substituindo o Pandas pelo **Apache Spark (PySpark)**.

- Instale o `pyspark` no Colab.
- Utilize `spark.read.parquet` para ler da Bronze.
- Reescreva as validações e filtros utilizando a sintaxe de Dataframes do Spark.
- Realize as agregações da Gold utilizando operações distribuídas.
- Compare o tempo de execução e a facilidade de leitura do código.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

O Processo ETL (Extract, Transform, Load)

O processo de ETL fundamentou-se na necessidade de integrar dados de sistemas transacionais heterogêneos para um repositório centralizado, tipicamente um Data Warehouse. Conforme define a documentação da Google Cloud (2024), este fluxo ocorre em três etapas sequenciais rígidas.

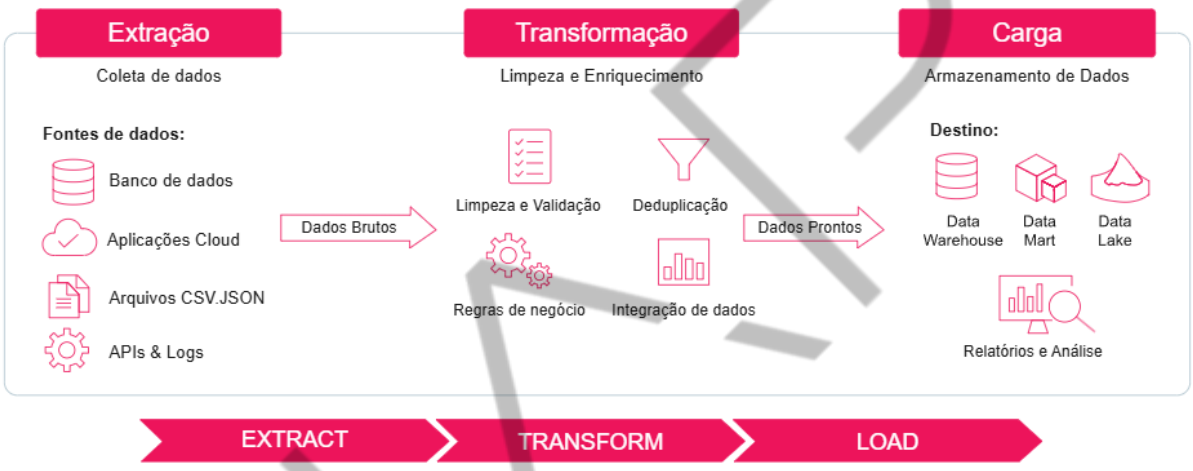


Figura 1 - Exemplificação de um pipeline ETL
Fonte: Elaborado pelo autor utilizando a ferramenta Canva IA (2025)

Inicialmente, na fase de **Extração**, os dados são capturados de fontes como CRMs (Salesforce), bancos relacionais (PostgreSQL, MySQL) ou arquivos planos. O desafio técnico aqui reside em não impactar a performance dos bancos de origem durante a leitura.

Nota: falhas na extração resultam em data drift ou missing values que podem degradar a performance do seu modelo.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

TIPOS DE EXTRAÇÃO	DESCRIÇÃO	VANTAGENS	DESVANTAGENS
Extração Completa (Full Extraction)	Todos os dados da fonte são extraídos em cada execução.	Simples de implementar, garante a integridade dos dados.	Alto consumo de recursos, lento para grandes volumes.
Extração Incremental (Incremental Extraction)	Apenas os dados que foram alterados ou adicionados desde a última extração são coletados.	Rápido, menor consumo de recursos, ideal para fontes em constante mudança.	Requer um mecanismo de rastreamento de alterações (timestamps, logs, *Change Data Capture - CDC*).

Tabela 1 – Tipos de extração
Fonte: elaborado pelo autor (2025)

Na etapa subsequente, a **Transformação**, os dados residem temporariamente em uma área de transição (staging area). É neste momento, fora do banco de dados final, que ocorrem as limpezas, padronizações de tipos, anonimização de dados sensíveis (PII) e agregações. Por ser realizada em um servidor de processamento dedicado, essa etapa costuma ser intensiva em recursos computacionais.

Por fim, ocorre a **Carga (Load)**, onde o dado já estruturado e "limpo" é inserido no destino final.

A principal característica do ETL, sob a ótica da Ciência de Dados, é que ele pressupõe que sabemos antecipadamente quais dados são importantes. Se uma variável for descartada durante a transformação para economizar espaço, ela é perdida, exigindo a recodificação do pipeline para recuperá-la — um gargalo frequente em projetos de Machine Learning exploratório.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

TIPOS DE CARGA	DESCRIÇÃO	APLICAÇÃO
Carga Inicial (Initial Load)	A primeira carga de dados. Envolve grandes volumes de dados históricos.	Configuração inicial do Data Warehouse.
Carga Incremental (Incremental Load)	Adicionar ou atualizar apenas os registros que foram transformados.	Manutenção diária ou horária do DW.
SCD (Slowly Changing Dimensions)	Estratégia para gerenciar dimensões que mudam ao longo do tempo (e.g., endereço de um cliente).	Essencial para manter o histórico analítico.

Tabela 2 – Tipos de Carga
Fonte: elaborado pelo autor (2025)

ESTRATÉGIAS DE CARGA	DESCRIÇÃO
Inserção/Atualização = Upsert	Insere registros novos e atualiza os registros já existentes.
Sobrescrita = Truncate and Load	Remove todos os dados da tabela e recarrega o conjunto completo do zero.

Tabela 3 – Tipos de estratégias para carga
Fonte: elaborado pelo autor (2025)

Comparação ETL vs. ELT e Cenários de Uso

Como mencionado, a principal diferença entre ETL e ELT está na ordem da transformação. No ETL tradicional, após extrair os dados das fontes, realiza-se a transformação em um servidor ou etapa intermediária e só então os dados já tratados são carregados no destino. Já no ELT, extrai-se os dados e carrega-se tudo sem transformações prévias no destino (por exemplo, um data lake na nuvem); a transformação é feita depois, dentro do próprio sistema de destino conforme a necessidade.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

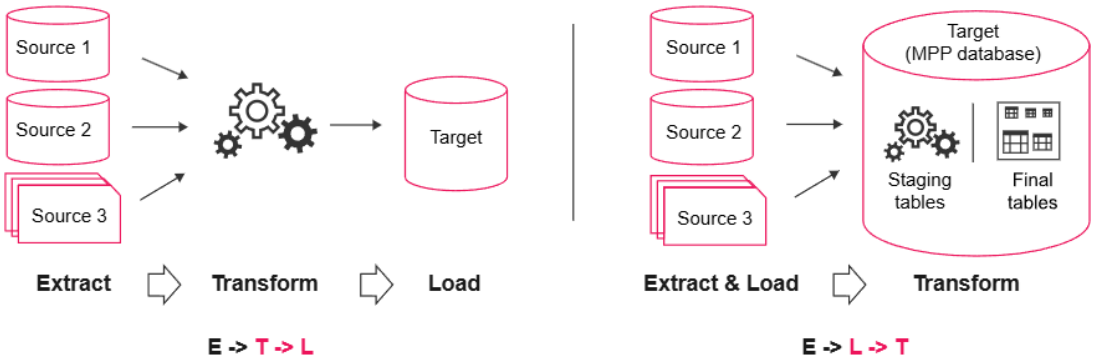


Figura 2 - Comparação entre ETL e ELT
Fonte: AMAZON WEB SERVICES (2024)

A decisão entre utilizar ETL ou ELT depende diretamente do contexto do projeto, das características dos dados e da infraestrutura disponível. Ambas as abordagens coexistem na prática e podem até ser combinadas, mas cada uma apresenta vantagens específicas conforme o cenário:

6. Volume de Dados
    - ELT é mais adequado quando o volume de dados é muito grande, pois aproveita o poder de processamento elástico de data warehouses e data lakes.
    - ETL funciona bem para volumes menores ou moderados, nos quais transformar antes de carregar não gera gargalos significativos.
7. Complexidade das Transformações
    - ETL geralmente é usado quando as regras de negócio são complexas, exigem lógica avançada ou precisam de um ambiente totalmente controlado antes da carga.
    - ELT é indicado quando as transformações podem ser realizadas diretamente no sistema de destino, frequentemente com SQL ou funções nativas do warehouse.
8. Características do Sistema de Destino

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

- Plataformas como BigQuery, Redshift e Snowflake favorecem ELT, pois foram projetadas para executar transformações internas em alta escala.
- Sistemas legados ou menos escaláveis podem exigir dados já prontos, tornando ETL a abordagem mais adequada.

Historicamente, o ETL dominou a integração de dados desde os anos 1970 (na era dos primeiros data warehouses) e era praticamente obrigatório devido às limitações de armazenamento e processamento da época. Hoje, entretanto, com o barateamento do armazenamento e a elasticidade da nuvem, tornou-se viável guardar dados brutos em grande volume e processá-los sob demanda. Em suma, ETL e ELT não se excluem, mas se complementam.

Vale também destacar que ETL geralmente é associado a cargas em lote (batch), executadas em intervalos (por exemplo, diariamente), ao passo que ELT permite cargas contínuas ou quase em tempo real, já que os dados brutos podem ser ingeridos imediatamente e transformados conforme a necessidade de consulta.

Essa diferença é importante: se a aplicação requer dados sempre atualizados ou análises em tempo real, arquiteturas ELT ou híbridas (com streaming) serão mais indicadas. Já se o foco é consistência absoluta dos dados antes do uso (por exemplo, em relatórios financeiros fechados mensalmente, que demandam dados completamente limpos e consolidados), um processo ETL pode fazer mais sentido.

Arquitetura de pipelines de dados: On-Premises vs. Cloud

Ao projetar um pipeline ETL, é essencial compreender o contexto de infraestrutura em que ele será executado. A forma como os dados são extraídos, transformados e carregados está diretamente ligada às capacidades do ambiente — seja ele local (on-premises) ou na nuvem.

As arquiteturas on-premises dependem inteiramente da infraestrutura física instalada no data center da empresa. De forma geral, seguem o seguinte fluxo:

- **Extração:** os dados são coletados de sistemas internos (ERPs, CRMs, bancos relacionais, arquivos locais etc.).

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

- **Transformação:** um **servidor ou cluster dedicado** executa scripts, rotinas ou ferramentas especializadas para limpeza, padronização e aplicação de regras de negócio.
- **Carga:** os dados tratados são armazenados em um banco corporativo ou data warehouse local, também hospedado em servidores da própria organização.

Esse modelo exige atenção constante à **capacidade de hardware**, pois todo o processamento depende de recursos físicos disponíveis.

Com a migração de aplicações e bancos de dados para a nuvem, os pipelines de dados também passaram a se beneficiar de serviços gerenciados e escalabilidade elástica. A arquitetura, nesse caso, segue uma dinâmica diferente:

- **Fontes e destinos** podem estar totalmente na nuvem (SaaS, bases cloud, data lakes).
- As etapas de processamento ocorrem em serviços **serverless** ou **clusters gerenciados**, que alocam os recursos automaticamente.
- Não é necessário configurar ou manter servidores manualmente: o provedor de nuvem se responsabiliza por infraestrutura, segurança e escalabilidade.

Na prática, muitas organizações adotam modelos híbridos, integrando sistemas locais com pipelines cloud para obter o melhor de cada abordagem.

Ferramentas e serviços ETL/ELT

Para operacionalizar as arquiteturas de dados e garantir a eficiência dos pipelines ETL e ELT, é necessário um conjunto de tecnologias que se consolidaram como padrões de mercado. A seleção adequada da linguagem de programação, do motor de processamento e do formato de armazenamento, impacta diretamente a performance, a escalabilidade e os custos operacionais da infraestrutura de dados em nuvem, mas perceba, essas escolhas vão variar muito de acordo com a sua necessidade de negócio e disponibilidade dos recursos no momento da execução.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Linguagens de Programação no Contexto de Dados

No cenário atual de Engenharia de Dados, as linguagens são selecionadas com base na sua capacidade de integração, processamento e orquestração:

- **Python:** além de ser predominante na orquestração (via Airflow), o Python consolidou-se como uma ferramenta robusta para a execução direta de transformações (ETL *code-first*). É comum que a etapa de transformação do ELT seja realizada inteiramente em Python, utilizando bibliotecas como Pandas para cargas menores ou APIs de processamento distribuído (como PySpark) para Big Data.
- **SQL e Spark SQL:** embora o SQL ANSI seja a base para consultas em Data Warehouses, no contexto de pipelines de dados, o destaque recai sobre o Spark SQL. Esta extensão permite integrar a sintaxe SQL diretamente em fluxos de processamento distribuídos. Diferente do uso tradicional focado apenas em consultas analíticas (*querying*) ou relatórios, o Spark SQL é utilizado estruturalmente dentro dos pipelines para realizar transformações pesadas, agregações e filtragens, unindo a simplicidade declarativa do SQL com a performance de *clusters* de computação.
- **Scala e Java:** são as linguagens fundamentais da infraestrutura de Big Data. Grande parte dos motores de processamento distribuído (como Apache Spark e Apache Kafka) é construída sobre a JVM (Java Virtual Machine). Embora o Python seja frequentemente usado como interface, o conhecimento destas linguagens é relevante para otimização de performance e *tuning* de clusters em cenários de alta volumetria.

Formatos de Arquivos e Serialização

A eficiência de um pipeline ETL não depende apenas do código, mas de como o dado é persistido no *Data Lake*. Formatos de texto simples (como CSV ou JSON), embora legíveis, são ineficientes para grandes volumes devido à falta de compressão e tipagem fraca. O mercado adota formatos binários otimizados para diferentes cargas de trabalho:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Formatos Colunares (Apache Parquet e ORC)

Para cargas de trabalho analíticas (OLAP), onde o pipeline precisa ler grandes volumes de histórico para gerar agregações, o **Apache Parquet** é o padrão industrial. Armazena dados organizados por coluna, e não por linha. Isso habilita o *columnar pruning*, permitindo que o sistema leia apenas as colunas necessárias para uma consulta, ignorando o restante do arquivo e reduzindo drasticamente o I/O. Além disso, suporta compressão eficiente (Snappy, Gzip) preservando o esquema dos dados.

Formatos Baseados em Linha (Apache Avro)

O **Apache Avro** é amplamente utilizado em pipelines de ingestão e sistemas de *streaming*. Armazena dados por linha e carrega o esquema (schema) no cabeçalho do próprio arquivo. É otimizado para operações de escrita intensa (*write-heavy*), sendo o formato padrão para serialização de mensagens em tópicos do Kafka, garantindo a evolução de esquema (*schema evolution*) sem quebrar consumidores a jusante.

Open Table Formats (Delta Lake, Iceberg e Hudi)

Para solucionar as limitações de transacionalidade em Data Lakes, surgiram os formatos de tabela modernos. Tecnologias como **Delta Lake** e **Apache Iceberg** adicionam uma camada de metadados transacionais sobre arquivos Parquet.

Impacto no Pipeline: wstes formatos trazem garantias ACID (*Atomicity, Consistency, Isolation, Durability*) para o Data Lake. Isso permite que pipelines realizem operações complexas de atualização (*Merge/Upsert*) e exclusão (*Delete*) de registros de forma segura e concorrente, além de permitir "viagem no tempo" (*Time Travel*) para auditoria e *rollback* de dados.

A Arquitetura de Camadas

Ao lidar com grandes volumes de dados em Data Lakes, Lakehouses ou Data Warehouses, é essencial estruturar o ambiente em camadas. Essa organização facilita governança, qualidade, desempenho e rastreabilidade.

A arquitetura mais utilizada para apoiar análises avançadas e Ciência de Dados é a **Medallion Architecture**, composta pelas camadas **Bronze, Silver e Gold**, às quais correspondem níveis progressivos de qualidade e refinamento.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

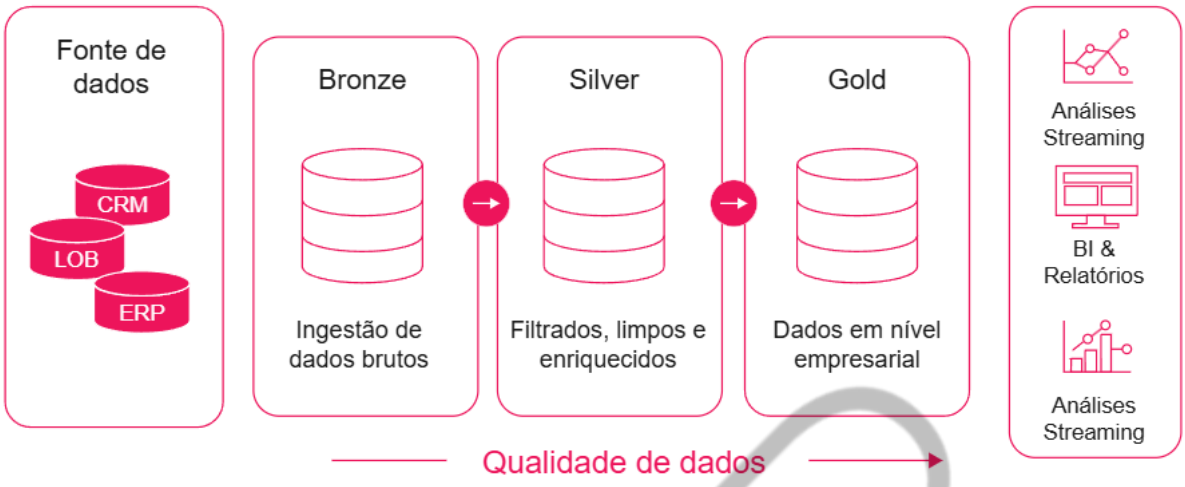


Figura 3 - Arquitetura Medalhão
Fonte: Elaborado pelo autor utilizando a ferramenta Canva IA (2025)

Camada de Ingestão (Dados Brutos/Bronze): aqui, os dados são capturados **exatamente como estão**, preservando o estado bruto das fontes originais. O objetivo é garantir que **nenhuma informação se perca**. Tudo que entra na plataforma é preservado para auditoria, reprocessamento e rastreabilidade.

Camada de Staging (Pré-Processamento/Buffer Técnico): a camada de staging funciona como uma **área intermediária e temporária**, onde os dados passam por verificações básicas antes das transformações mais profundas. Em muitas arquiteturas, a camada de staging é temporária: após os dados serem processados para a camada Bronze/Silver, arquivos intermediários podem ser descartados.

Camada de Transformação/Curadoria (Silver): na camada de curadoria ocorre o processamento dos dados, onde eles são transformados, limpos e integrados para gerar conjuntos confiáveis. Essa camada é fundamental para garantir que os dados avancem para o consumo com qualidade adequada. Aqui também se mantém o **lineage**, preservando o vínculo entre o dado refinado e suas origens.

Camada de Armazenamento para Consumo (Serviço/Gold): a camada Gold representa os dados **finalizados e prontos para uso**, modelados para análise, BI, exploração ad hoc ou treinamento de modelos.

Camada Semântica (Modelo de Negócio): acima da camada Gold, muitas arquiteturas modernas implementam uma **camada semântica**, criada para facilitar o entendimento do dado pelos usuários de negócio. Implementações típicas incluem

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

modelos tabulares, semantic layers nativos (como no Looker), views lógicas ou catálogos especializados.

Após a definição das camadas, é válido mencionar que os pipelines ETL/ELT transitam através dessas camadas de forma ordenada. Ou seja, um pipeline típico irá fazer a ingestão de dados na camada bruta, movê-los (ou copiá-los) para a área de staging, então transformá-los gerando dados curados (camada intermediária), e finalmente carregá-los na camada de serviço (consumo) já prontos para uso.

Dimensões da Qualidade de Dados e sua Aplicação em Pipelines ETL/ELT

A qualidade dos dados é um elemento fundamental para qualquer solução analítica ou de Inteligência Artificial. Modelos, relatórios e decisões só são tão bons quanto os dados que os alimentam. Por isso, compreender como medir e garantir qualidade ao longo de um pipeline ETL/ELT é essencial. As dimensões mais amplamente utilizadas incluem:

- **Completude**: avalia se todos os campos e registros necessários estão presentes. Dados faltantes reduzem a confiabilidade das análises e podem introduzir vieses.
- **Precisão (Acurácia)**: refere-se ao quanto os dados refletem corretamente a realidade. Valores incorretos – como endereço errado ou idade incoerente – comprometem diretamente modelos e relatórios.
- **Consistência**: verifica se os dados mantêm coerência entre sistemas e ao longo do tempo. Informações divergentes em bases diferentes indicam falhas de governança e sincronização.
- **Validade (Conformidade)**: avalia se os dados seguem regras, formatos e padrões estabelecidos. Exemplos incluem validação de formato de data, domínios de valores ou padrões de telefone.
- **Atualidade (Pontualidade)**: verifica se os dados estão atualizados e disponíveis no momento adequado. Dados atrasados reduzem o valor analítico e podem levar a decisões baseadas em informações defasadas.
- **Unicidade**: garante que cada entidade apareça apenas uma vez. Duplicidades distorcem métricas, análises e processos operacionais.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Outras dimensões podem ser consideradas, como integridade referencial e adequação ao propósito, mas as seis acima formam uma base sólida para diagnóstico e monitoramento contínuo da saúde dos dados.

A qualidade dos dados deve ser tratada como um ciclo contínuo. Cada dimensão, completude, precisão, consistência, validade, atualidade e unicidade, atua como uma lente para identificar riscos e oportunidades de melhoria dentro do pipeline. Sua aplicação pode ser resumida da seguinte forma:

ETAPA DO PIPELINE	FOCO DA QUALIDADE	EXEMPLO DE AÇÕES
Extração	Completude, fidelidade	Contagens, checksums, validação de campos obrigatórios
Transformação	Consistência, validade, unicidade, acurácia	Deduplicação, padronização, limpeza, validações automatizadas
Carga	Integridade, unicidade, consistência	Verificação pós-carga, análise de rejeições, auditorias

Tabela 4 – Resumo das etapas de qualidade de dados aplicadas no ETL
Fonte: elaborado pelo autor (2025)

Conclusão

A decisão entre ETL e ELT, bem como a escolha da ferramenta, impacta diretamente a arquitetura e performance da solução.

A arquitetura moderna de dados tende a ser híbrida, utilizando o paradigma ELT para popular Data Lakes (camadas Bronze/Silver) onde os(as) cientistas têm liberdade para explorar, e processos mais rígidos para a camada Gold, que serve para relatórios.

Para o(a) AI Scientist, dominar os conceitos de camadas de dados e entender a infraestrutura subjacente não significa substituir o(a) Engenheiro(a) de Dados. Significa possuir a autonomia necessária para desenhar soluções que sejam não apenas acuradas academicamente, mas tecnicamente viáveis, escaláveis e resilientes no mundo real.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

Entre as principais tendências, destaca-se o crescimento do **Analytics Engineering**, que aproxima cientistas de dados da camada de transformação por meio de ferramentas como dbt, promovendo versionamento, testes e modularização de transformações em SQL.

Outra tendência relevante é a adoção de **Feature Stores**, que conectam pipelines ELT à reutilização e padronização de atributos para modelos de Machine Learning.

Paralelamente, práticas de **Data Quality as Code** e **Data Observability** vêm sendo incorporadas aos pipelines, permitindo monitoramento contínuo da saúde dos dados, semelhante ao que já ocorre em observabilidade de software.

Indicação de artigos e vídeos:

“Data engineering for ML and AI with AWS analytics” – AWS re:Invent 2024 (ANT405) - Youtube:

https://www.youtube.com/watch?v=bezVijrUztA

Indicação de livros:

“Designing Data-Intensive Applications” – Martin Kleppmann: discute os fundamentos da construção de sistemas de dados escaláveis, confiáveis e distribuídos. O autor aborda conceitos essenciais como ingestão de dados, processamento em batch e streaming, entre outros.

“Fundamentals of Data Engineering: Plan and Build Robust Data Systems” – Joe Reis e Matt Housley: apresenta uma visão moderna e prática da engenharia de dados. Os autores exploram pipelines ETL e ELT, arquiteturas em nuvem, modelagem em camadas, qualidade de dados e governança, sempre conectando esses conceitos às necessidades de análises avançadas e Inteligência Artificial.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você revisitou os fundamentos que estruturam qualquer solução de Inteligência Artificial moderna, entendendo como os pipelines de dados sustentam a criação, o treinamento e a operacionalização de modelos. Ao longo do conteúdo, você compreendeu a diferença entre as arquiteturas ETL e ELT, reconhecendo que o ETL concentra o processamento antes da carga e se ajusta bem a ambientes tradicionais, enquanto o ELT desloca a transformação para o Data Lake ou Data Warehouse, favorecendo maior flexibilidade, acesso ao dado bruto e escalabilidade. Observou também as etapas essenciais que compõem um pipeline de dados, passando pela extração, transformação e pela carga.

Além disso, aprendeu como a evolução das arquiteturas em nuvem possibilitou o processamento em larga escala, permitindo que cientistas de dados trabalhem com ambientes híbridos que combinam Data Lakes, Data Warehouses e ferramentas cloud-native. A modelagem em camadas, estruturada nas etapas Raw, Bronze, Silver e Gold, mostrou como os dados passam por diferentes níveis de maturidade, desde o estado bruto até métricas finais prontos para uso estratégico ou para alimentar modelos em produção. Esse entendimento reforça a importância do desenho arquitetural para atender desde a experimentação inicial até processos robustos e governados de Machine Learning.

Outro ponto abordado foi a relevância da qualidade de dados como determinante da qualidade dos modelos. Encerrando o conteúdo, deve estar claro que dominar esses fundamentos concede ao AI Scientist a capacidade de interpretar, colaborar e construir pipelines de dados confiáveis, escaláveis e adequados ao ciclo de vida completo de modelos de Inteligência Artificial.

Mapa Mental

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

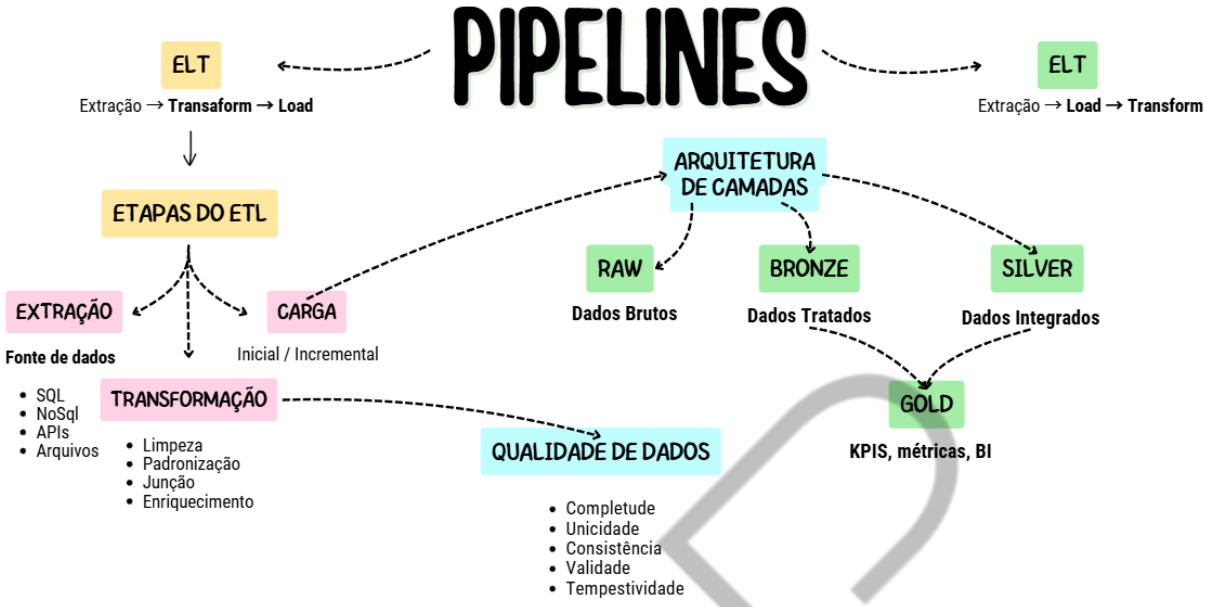


Figura 4 - Mapa mental com o conteúdo abordado

Fonte: Elaborado pelo autor utilizando a ferramenta Canva IA (2025)

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

AMAZON WEB SERVICES. **Comparando os serviços de ETL para extração, transformação e carga de dados na AWS**. AWS Blog Brasil, 2023. Disponível em: https://aws.amazon.com/pt/blogs/aws-brasil/comparando-os-servicos-de-etl-para-extracao-transformacao-e-carga-de-dados-na-aws/. Acesso em: 05 dez. 2025.

AMAZON WEB SERVICES. **The Difference Between ETL and ELT**. AWS Compare, 2024. Disponível em: https://aws.amazon.com/pt/compare/the-difference-between-etl-and-elt/. Acesso em: 05 dez. 2025.

AMAZON WEB SERVICES. **What is ETL?**. AWS What Is, 2024. Disponível em: https://aws.amazon.com/pt/what-is/etl/. Acesso em: 05 dez. 2025.

APACHE SOFTWARE FOUNDATION. **Apache Parquet**. 2024. Disponível em: https://parquet.apache.org/.

CASTELLANOS, C. C.; TORRES, G. G. (2021). *Comparando os serviços de ETL para extração, transformação e carga de dados na AWS*. Blog da AWS – AWS Brasil.

GOOGLE CLOUD. **Executar um pipeline de ETL**. Documentação do Google Cloud Shell, 2024. Disponível em: https://docs.cloud.google.com/shell/docs/cloud-shell-tutorials/deploystack/etl-pipeline?hl=pt-br. Acesso em: 05 dez. 2025.

GOOGLE CLOUD. **O que é ETL?**. Google Cloud Learn, 2024. Disponível em: https://cloud.google.com/learn/what-is-etl?hl=pt-BR. Acesso em: 05 dez. 2025.

REIS, J.; HOUSLEY, M. **Fundamentals of Data Engineering**. O'Reilly Media, 2022.

SERRA, James. **Deciphering Data Architectures**: Choosing Between a Modern Data Warehouse, Data Fabric, Data Lakehouse, and Data Mesh. O'Reilly Media, 2024. (Referência conceitual baseada no tópico "Decifrando Arquiteturas de Dados").

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

Pipelines. Arquitetura. Qualidade.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com